🐍 Python — Complete Interview Prep Guide

Basic → Advanced | 5.5 YOE Java Dev Level

Har topic: Definition (English + Hinglish) + Deep Example + Interview Q&A

📖 GLOSSARY — Python Ke Unique Concepts

Word	Matlab
Dynamic typing	Variable ka type runtime pe decide — pehle declare nahi karna (Java mein `int x = 5`, Python mein sirf `x = 5`)
Duck typing	"Agar quack karta hai toh duck hai" — type check nahi, behavior check karo
Mutable	Object create hone ke baad change ho sakta hai — list, dict, set
Immutable	Change nahi hota create hone ke baad — str, tuple, int
Iterable	Jisme se ek ek item nikaal sako — list, dict, string
Iterator	Iterable ka cursor — `next()` call pe next item deta hai
Generator	Lazy iterator — `yield` se ek ek value deta hai, sab ek saath nahi
Decorator	`@something` — function ko wrap karta hai extra behavior add karne ke liye
Context Manager	`with` statement — setup aur cleanup automatic (Java ka try-with-resources)
First-class fn	Function ko variable mein store karo, argument mein pass karo
Closure	Inner function jo outer function ke variables yaad rakhta hai
Comprehension	Ek line mein list/dict/set banana — `[x*2 for x in range(10)]`
GIL	Global Interpreter Lock — ek waqt ek Python thread truly parallel nahi

---

📌 TOPIC 1: Variables aur Data Types ⭐ ⭐ ⭐

Dynamic Typing — Java Se Kya Fark

Java mein har variable ka type declare karna padta hai. Python mein type runtime pe decide hoti hai — aur change bhi ho sakti hai.

```python
# Java style: String name = "Rahul"  ← type pehle batao
# Python style:
name    = "Rahul"       # Python khud samjha — ye string hai
age     = 25            # int
pi      = 3.14          # float
flag    = True          # bool — capital T! (Java mein lowercase)
nothing = None          # None — Java ka null

x = 10                  # x is int
x = "hello"             # ab x is str — Java mein ye ERROR deta
x = [1, 2, 3]           # ab x is list — Python allow karta hai

# Type check
print(type(name))           # <class 'str'>
print(isinstance(age, int)) # True — Java ka instanceof

# Multiple assignment
a, b, c = 1, 2, 3      # unpacking
a, b    = b, a          # swap — koi temp variable nahi chahiye!
```

Data Types — Sab Important Wale

1. String (str) — Immutable

```python
```

```python
name = "Rahul"

# Common methods
name.upper()            # "RAHUL"
name.lower()            # "rahul"
name.strip()            # whitespace remove dono ends se
name.split(",")         # list mein tod do
",".join(["a","b"])     # list → string — "a,b"
name.replace("R","r")   # "rahul"
name.startswith("Ra")   # True
len(name)               # 5

# f-string — sabse important!
city  = "Delhi"
intro = f"Main {name} hoon, {city} se"

# Expressions bhi chal sakte hain:
result = f"2 + 2 = {2 + 2}"         # "2 + 2 = 4"
upper  = f"{name.upper()} rocks!"   # "RAHUL rocks!"

# String immutable hai — direct change nahi ho sakta
s = "hello"
# s[0] = "H"   # ERROR! String immutable hai
s = "Hello"    # naya string banana padega
```

2. Integer aur Float

```python
x = 10      # int — Python mein unlimited size (no overflow!)
y = 3.14    # float — 64-bit double precision

# Division — Java se ALAG hai!
10 / 3      # 3.333...   (Java mein int/int = 3 hota)
10 // 3     # 3          floor division
10 % 3      # 1          remainder
2 ** 10     # 1024       power (Java mein Math.pow())

# Type conversion
int("42")       # 42
float("3.14")   # 3.14
str(100)        # "100"
```

3. Boolean

```python
```

```
is_ready = True    # Capital T — Java ka true nahi!
is_done  = False   # Capital F

# Truthy / Falsy — important concept
# Yeh sab False ki tarah behave karte hain:
# False, None, 0, 0.0, "", [], {}, set()

items = []
if items:           # empty list = falsy — seedha check karo
    print("items hain")
else:
    print("koi item nahi")   # ye chalega
```

4. List — Mutable, Ordered (Java ka ArrayList)

```python
fruits = ["apple", "banana", "cherry"]
mixed  = [1, "hello", True, 3.14]    # alag types ek saath OK

# Access
fruits[0]       # "apple"   — 0 indexed
fruits[-1]      # "cherry"  — last element!
fruits[1:3]     # ["banana", "cherry"]  — slicing

# Common operations
fruits.append("mango")      # end mein add
fruits.insert(1, "grape")   # position pe add
fruits.remove("banana")     # value se remove
fruits.pop()                # last element nikalo + remove
fruits.sort()               # sort karo (in-place)
len(fruits)                 # size
```

5. Tuple — Immutable List

```python
```

```python
coords = (28.6, 77.2)   # immutable — change nahi ho sakta
rgb    = (255, 128, 0)

# Unpacking — bahut useful
lat, lng = coords       # seedha unpack!
r, g, b  = rgb

# Function se multiple values return karna
def get_user():
    return "Rahul", 25, "Delhi"   # tuple return
name, age, city = get_user()       # unpack karo
```

6. Dictionary — Key-Value (Java ka HashMap)

```python
user = {
    "name":   "Rahul",
    "age":    25,
    "skills": ["Python", "Java"],
}

user["name"]                # "Rahul" — KeyError agar key nahi
user.get("age")             # 25 — safe access
user.get("phone", "N/A")    # "N/A" — default value

user["city"] = "Delhi"      # naya key add
user["age"]  = 26           # existing update

user.keys()                 # dict_keys([...])
user.values()               # dict_values([...])
user.items()                # dict_items([("name","Rahul"),...])

"email" in user             # True/False — key exist karta hai?
del user["age"]             # delete karo
user.pop("city", None)      # safe delete
```

7. Set — Unique Values Only

```python
```

```python
tags = {"python", "java", "python"}   # duplicate remove!
print(tags)   # {"python", "java"}

a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
a | b    # Union:        {1,2,3,4,5,6}
a & b    # Intersection: {3,4}
a - b    # Difference:   {1,2}
```

❓ Interview Questions — Topic 1

Q: Python mutable aur immutable mein kya fark hai? Examples do.
A: Mutable = create ke baad change ho sakta hai — list, dict, set. Immutable = change nahi — str, int, tuple. Important kyunki function mein mutable object pass karo toh function change kar sakta hai (side effect). Immutable safe hai.

Q: Dynamic typing kya hai? Java se kaise alag?
A: Dynamic = type runtime pe decide hoti hai, pehle declare nahi karna. `x=5` (int), phir `x="hello"` (str) — valid. Java mein `int x=5`, phir `x="hello"` compile error deta. Fayda: fast development. Nuksan: runtime errors, IDE support kam.

Q: `is` vs `==` mein kya difference hai?
A: `==` value compare karta hai. `is` same object compare karta hai (memory address). `a=[1,2], b=[1,2]`: `a==b` True (same value), `a is b` False (different objects). None check hamesha `is None` se karo.

Q: List aur Tuple mein kya fark hai?
A: List mutable hai — create ke baad change ho sakti hai. Tuple immutable hai — change nahi hota. Tuple function se multiple values return karne ke liye use karo. Performance-wise tuple thoda faster aur memory efficient.

📌 TOPIC 2: Control Flow ⭐ ⭐

If / Elif / Else

```python
```

```python
score = 85

if score >= 90:
    grade = "A"
elif score >= 70:
    grade = "B"     # ye chalega
elif score >= 50:
    grade = "C"
else:
    grade = "F"

# Ternary (one-liner) — Java ka (cond ? a : b)
result = "Pass" if score >= 50 else "Fail"
```

For Loop

```python
```

```python
# Python mein for = for-each (Java ka enhanced for)
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)

# range() — numbers ke liye
for i in range(5):          # 0,1,2,3,4
    print(i)
for i in range(1, 6):       # 1,2,3,4,5
    print(i)
for i in range(0, 10, 2):   # 0,2,4,6,8  (step 2)
    print(i)

# enumerate() — index + value dono chahiye
for index, fruit in enumerate(fruits):
    print(f"{index}: {fruit}")
# 0: apple
# 1: banana

# zip() — do lists ek saath iterate
names  = ["Rahul", "Priya", "Amit"]
scores = [90, 85, 78]
for name, score in zip(names, scores):
    print(f"{name}: {score}")

# break aur continue
for i in range(10):
    if i == 3: continue     # 3 skip karo
    if i == 7: break        # 7 pe loop band
    print(i)    # 0,1,2,4,5,6
```

While Loop

python

```python
count = 0
while count < 5:
    print(count)
    count += 1

# Retry pattern — production mein common
retry = 0
while retry < 3:
    try:
        result = call_api()
        break
    except Exception:
        retry += 1
```

❓ Interview Questions — Topic 2

Q: `range(5)` kya return karta hai?
A: range object return karta hai — 0,1,2,3,4 values lazily. List nahi hai directly. `list(range(5))` se list banta hai. Memory efficient hai bade ranges ke liye.

Q: `enumerate()` kab use karte ho?
A: Jab for loop mein index aur value dono chahiye. `for i, val in enumerate(lst)` — instead of keeping a manual counter.

📌 TOPIC 3: Functions ⭐ ⭐ ⭐ ⭐

Basic Function + Type Hints

```python
def greet(name: str) -> str:
    """
    Docstring — function ka description.
    name: user ka naam
    Returns: greeting string
    """
    return f"Namaste, {name}!"

result = greet("Rahul")   # "Namaste, Rahul!"
```

Default Parameters

```python
```

```python
def create_config(model="gpt-4", temperature=0.7, max_tokens=1000):
    return {"model": model, "temp": temperature, "tokens": max_tokens}

create_config()                     # sab defaults
create_config("gpt-3.5")            # model override
create_config(temperature=0.0)      # sirf temperature override
create_config("claude", 0.5, 500)   # sab override

# ⚠️ GOTCHA — mutable default = bug!
def bad_append(item, lst=[]):       # ← shared across calls!
    lst.append(item)
    return lst

def good_append(item, lst=None):    # ← correct way
    if lst is None:
        lst = []
    lst.append(item)
    return lst
```

> 💡 Mutable default argument ek common interview trap hai!

*args aur **kwargs

```python
```

```python
# *args — variable positional arguments → tuple ban jaata hai
def total(*numbers):
    return sum(numbers)   # numbers is a tuple inside

total(1, 2)           # 3
total(1, 2, 3, 4, 5)  # 15
total()               # 0

# **kwargs — variable keyword arguments → dict ban jaata hai
def configure(**settings):
    print(settings)   # {"key": "value", ...}

configure(model="gpt-4", temp=0.7)   # dict pass

# Dono saath
def flexible(*args, **kwargs):
    print(args)     # (1, 2, 3)
    print(kwargs)   # {"name": "Rahul"}

flexible(1, 2, 3, name="Rahul")

# Unpacking when calling
def add(a, b, c): return a+b+c
nums = [1, 2, 3]
add(*nums)              # add(1,2,3) — list unpack
cfg = {"a":1,"b":2,"c":3}
add(**cfg)              # add(a=1,b=2,c=3) — dict unpack
```

First-Class Functions + Lambda

```python
```

```python
# Python mein functions = objects — variable mein store, pass kar sako
def double(x): return x * 2
def triple(x): return x * 3

operation = double         # function variable mein
operation(5)               # 10

def apply(func, numbers):  # function argument mein pass
    return [func(n) for n in numbers]

apply(double, [1,2,3,4])   # [2,4,6,8]
apply(triple, [1,2,3,4])   # [3,6,9,12]

# Lambda — anonymous one-liner function
# Java: x -> x * 2
square = lambda x: x * x
add    = lambda x, y: x + y

# Practical — sort with custom key
people = [{"name":"Rahul","age":25}, {"name":"Priya","age":22}]
people.sort(key=lambda p: p["age"])   # age se sort
```

Scope — Local, Global, Nonlocal

```python
count = 0   # global

def increment():
    global count      # global keyword — bahar wala modify karo
    count += 1

# Closure — inner function outer ke variables yaad rakhta hai
def make_counter(start=0):
    count = start
    def counter():
        nonlocal count    # outer scope modify karo
        count += 1
        return count
    return counter

c = make_counter(10)
c()   # 11
c()   # 12   — count yaad hai!
```

❓ Interview Questions — Topic 3

Q: `*args` aur `**kwargs` mein kya fark hai?
A: `*args` = variable positional arguments, tuple mein aate hain. `**kwargs` = variable keyword arguments, dict mein aate hain. Use karo jab pata nahi kitne args aayenge.

Q: Lambda function kab use karte ho?
A: One-liner anonymous functions ke liye — jaise `sort` ka `key=`, map/filter mein, ya jab ek baar use karna ho. Complex logic ke liye regular `def` better hai.

Q: Closure kya hota hai? Example do.
A: Inner function jo outer function ke variables yaad rakhta hai — even after outer function return ho jaaye. `make_counter()` example — `count` variable closure mein live karta hai. Factory functions banane ke liye useful.

Q: Mutable default argument bug kya hai?
A: `def f(lst=[])` — yeh list ek baar banti hai aur sab calls share karti hain. Isliye `def f(lst=None)`, phir `lst=[]` inside the function — yeh correct pattern hai.

📌 TOPIC 4: List Comprehensions ⭐⭐⭐⭐

Java ka `stream().map().filter().collect()` jaisa — but simpler aur concise.

Basic Comprehension

```python
# Old way — for loop
squares = []
for i in range(10):
    squares.append(i * i)

# New way — list comprehension
squares = [i * i for i in range(10)]
# [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

# Formula: [expression FOR item IN iterable]
```

With Condition (Filter)

```python
```

```python
# Formula: [expression FOR item IN iterable IF condition]

evens = [i for i in range(20) if i % 2 == 0]
# [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]

# Only passing students
students = [("Rahul",85), ("Priya",45), ("Amit",72)]
passed = [name for name, score in students if score >= 50]
# ["Rahul", "Amit"]
```

Dict Comprehension

```python
# Formula: {key: value FOR item IN iterable}

words   = ["apple", "banana", "cherry"]
lengths = {word: len(word) for word in words}
# {"apple": 5, "banana": 6, "cherry": 6}

# Existing dict transform
config       = {"model": "gpt-4", "temp": 0.7}
UPPER_CONFIG = {k.upper(): v for k, v in config.items()}
# {"MODEL": "gpt-4", "TEMP": 0.7}
```

Set Comprehension

```python
# Formula: {expression FOR item IN iterable}
numbers = [1, 2, 2, 3, 3, 4]
unique  = {x for x in numbers}   # {1, 2, 3, 4}
```

Generator Expression — Memory Efficient

```python
# () use karo [] ki jagah — lazy evaluation
squares_list = [x*x for x in range(1_000_000)]   # sab RAM mein!
squares_gen  = (x*x for x in range(1_000_000))   # sirf object

# sum() ke saath — efficient
total = sum(x*x for x in range(1_000_000))        # memory efficient!
```

> 💡 Generator expression prefer karo jab sirf ek baar iterate karna ho ya data bada ho.

❓ Interview Questions — Topic 4

Q: List comprehension kya hai? Normal loop se better kyun?
A: Concise way to create lists — ek line mein. Readable aur Pythonic. Thoda faster bhi hai append se. `[x*2 for x in nums]` vs manual loop with append.

Q: `[x for x in range(10) if x%2==0]` kya return karega?
A: `[0, 2, 4, 6, 8]` — 0 se 9 tak ke even numbers.

Q: List comprehension vs Generator expression — kab kaunsa?
A: List = result turant chahiye, multiple baar iterate karna ho. Generator = large data, memory concern, sirf ek baar iterate karna ho. `()` vs `[]`.

📌 TOPIC 5: Decorators ⭐⭐⭐⭐⭐

Function jo doosre function ko wrap kare — extra behavior add kare. Java ka `@Annotation` jaisa dikhta hai lekin kaam alag hai.

Decorator Andar Se Kaise Kaam Karta Hai

```python
# Step 1 — manually wrap karna
def logger(func):
    def wrapper(*args, **kwargs):
        print(f"  {func.__name__} shuru hua")
        result = func(*args, **kwargs)   # original call
        print(f"  {func.__name__} khatam hua")
        return result
    return wrapper

def say_hello():
    print("Hello!")

say_hello = logger(say_hello)   # manually wrap
say_hello()
# say_hello shuru hua
# Hello!
# say_hello khatam hua

# Step 2 — @ syntax shortcut (same kaam karta hai!)
@logger
def say_hello():
    print("Hello!")

# @logger  ↔  say_hello = logger(say_hello)
```

Timer Decorator — Practical Example

```python
import time
from functools import wraps

def timer(func):
    @wraps(func)   # original naam aur docstring preserve karo
    def wrapper(*args, **kwargs):
        start  = time.time()
        result = func(*args, **kwargs)
        end    = time.time()
        print(f"{func.__name__} took {(end-start)*1000:.2f}ms")
        return result
    return wrapper

@timer
def slow_function():
    time.sleep(0.5)
    return "done"

slow_function()   # slow_function took 500.xx ms
```

Decorator with Arguments

```python
def repeat(times):          # outer — argument leta hai
    def decorator(func):    # middle — function leta hai
        @wraps(func)
        def wrapper(*args, **kwargs):
            for _ in range(times):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator        # decorator return karo

@repeat(times=3)
def greet(name):
    print(f"Hello {name}!")

greet("Rahul")   # teen baar print hoga
```

@property Decorator

```python
```

```python
# Java ka getter/setter jaisa — but cleaner syntax
class Circle:
    def __init__(self, radius):
        self._radius = radius

    @property
    def radius(self):           # getter — attribute jaisa access
        return self._radius

    @radius.setter
    def radius(self, value):    # setter — validation ke saath
        if value < 0:
            raise ValueError("Radius negative nahi ho sakta")
        self._radius = value

    @property
    def area(self):             # computed property
        return 3.14159 * self._radius ** 2

c = Circle(5)
print(c.radius)   # 5   — getter call hua (method nahi lagta!)
print(c.area)     # 78.5 — auto compute
c.radius = 10     # setter call hua
c.radius = -1     # ValueError!
```

@staticmethod aur @classmethod

```python
```

```python
class BankAccount:
    interest_rate = 0.05   # class variable

    def __init__(self, owner, balance):
        self.owner   = owner
        self.balance = balance

    @staticmethod
    def validate_amount(amount):    # self nahi chahiye
        return amount > 0           # Java ka static method

    @classmethod
    def create_savings(cls, owner): # cls = class itself
        return cls(owner, 1000)     # factory method pattern

    def deposit(self, amount):
        if BankAccount.validate_amount(amount):   # static call
            self.balance += amount

# Use
BankAccount.validate_amount(500)          # True — class se call
acc = BankAccount.create_savings("Rahul") # factory
```

❓ Interview Questions — Topic 5

Q: Decorator kya hota hai? Andar se kaise kaam karta hai?
A: Function jo doosre function ko wrap kare extra behavior add karne ke liye. `@logger` ek shortcut hai `logger(func)` ke liye. Wrapper function original ka naam preserve kare isliye `@wraps` use karo.

Q: `@wraps` kyun use karte ho?
A: Bina `@wraps` ke decorated function ka `__name__` aur `__doc__` wrapper ka ho jaata hai. `@wraps` original function ka naam aur docstring preserve karta hai — debugging aur introspection ke liye important.

Q: `@property` vs regular method — kya difference?
A: `@property` se method ko attribute jaisa access kar sakte ho — `c.area` (brackets nahi). Regular method mein `c.area()` call karna padta. `@property` computed values aur validation ke liye achha hai.

Q: `@staticmethod` aur `@classmethod` mein kya fark hai?
A: `@staticmethod` = neither self nor cls — sirf utility function, class ke saath logically related. `@classmethod` = `cls` milta hai — factory methods banane ke liye, subclassing ke saath kaam karta hai.

📌 TOPIC 6: Generators aur yield ⭐⭐⭐⭐⭐

Generator = lazy iterator. Normal function ek baar mein sab return karta hai. Generator `yield` se ek ek value deta hai — pause karke. Memory efficient + large data ke liye perfect.

Normal Function vs Generator

```python
# Normal function — sab ek saath memory mein
def get_squares(n):
    result = []
    for i in range(n):
        result.append(i * i)
    return result   # pura list return

squares = get_squares(1_000_000)   # 1M numbers — RAM mein!

# Generator — ek ek karke
def square_gen(n):
    for i in range(n):
        yield i * i    # yield = value do aur PAUSE karo

gen = square_gen(1_000_000)   # sirf generator object bana
next(gen)   # 0  — first value
next(gen)   # 1  — second value
next(gen)   # 4  — third value

# for loop ke saath — automatically next() call karta hai
for square in square_gen(10):
    print(square)   # 0, 1, 4, 9, 16...
```

yield Kaise Kaam Karta Hai — Step by Step

```python
```

```python
def countdown(n):
    print(f"Start: {n} se")   # pehli next() pe chalega
    while n > 0:
        yield n                # yahan PAUSE ho jaata hai
        n -= 1                 # next next() pe yahan se continue
    print("Khatam!")           # generator exhaust hone pe

c = countdown(3)
print("Generator banaya — abhi kuch nahi chala!")
next(c)   # "Start: 3 se" print, phir 3 return
next(c)   # 2
next(c)   # 1
next(c)   # "Khatam!" print, phir StopIteration raise
```

Practical Use Cases

```python
```

```python
# 1. Large file reading — ek ek line
def read_large_file(filepath):
    with open(filepath) as f:
        for line in f:
            yield line.strip()   # ek ek line yield karo

# Memory efficient — poori file ek saath load nahi hoti
for line in read_large_file("huge_log.txt"):
    process(line)

# 2. Infinite sequence
def fibonacci():
    a, b = 0, 1
    while True:      # infinite loop — but lazy
        yield a
        a, b = b, a + b

fib = fibonacci()
for _ in range(10):
    print(next(fib))   # 0,1,1,2,3,5,8,13,21,34

# 3. Pipeline — generators chain karo
def read_data(data):
    for item in data:
        yield item

def filter_positive(gen):
    for item in gen:
        if item > 0:
            yield item

def square_it(gen):
    for item in gen:
        yield item * item

data = [-1, 2, -3, 4, 5]
pipeline = square_it(filter_positive(read_data(data)))
list(pipeline)   # [4, 16, 25]
```

❓ Interview Questions — Topic 6

Q: Generator aur normal function mein kya fark hai?
A: Normal function = ek baar mein sab return. Generator = `yield` se ek ek value lazily. Generator memory efficient hai — large data ke liye. `return` vs `yield` — return end karta hai, yield pause karta hai.

Q: StopIteration kya hai?
A: Jab generator exhaust ho jaata hai (aur values nahi) toh `StopIteration` raise hota hai. `for` loop automatically ise handle karta hai. Manual `next()` use karne pe `try/except` lagao.

Q: Generator expression kya hai?
A: `(x*x for x in range(10))` — list comprehension jaisa but `()` use karo. Lazy evaluation. `sum()`, `max()`, `min()` ke saath directly pass kar sako — memory efficient.

📌 TOPIC 7: Context Managers ⭐⭐⭐⭐

`with` statement ke saath kaam karta hai. Setup automatically hota hai (`__enter__`) aur cleanup guaranteed hota hai (`__exit__`) — error aaye ya na aaye. Java ka try-with-resources same concept hai.

Problem Without Context Manager

```python
# Risky — agar error aaya toh file close nahi hogi
file = open("data.txt", "r")
try:
    content = file.read()
finally:
    file.close()   # hamesha close karo

# Context manager — cleaner!
with open("data.txt", "r") as file:
    content = file.read()
# File automatically close ho gayi — with block ke baad
# Error aaye tab bhi close hogi — guaranteed!
```

Apna Context Manager — Class Method

```python
```

```python
class DatabaseConnection:
    def __init__(self, host, db_name):
        self.host    = host
        self.db_name = db_name
        self.conn    = None

    def __enter__(self):            # with block shuru
        print(f"Connecting to {self.host}/{self.db_name}")
        self.conn = create_connection(self.host, self.db_name)
        return self.conn            # "as" clause mein milega

    def __exit__(self, exc_type, exc_val, exc_tb):
        # exc_type = None agar koi error nahi
        if exc_type is not None:
            print(f"Error: {exc_val}, rolling back")
            self.conn.rollback()
        else:
            self.conn.commit()
        self.conn.close()           # hamesha close
        return False                # exception suppress mat karo

with DatabaseConnection("localhost", "mydb") as conn:
    conn.execute("SELECT * FROM users")
# Auto commit/rollback + close guaranteed!
```

@contextmanager — Simpler Way

```python
```

```python
from contextlib import contextmanager

@contextmanager
def timer_context(label):
    import time
    start = time.time()
    print(f"{label} shuru")
    try:
        yield    # with block ka code yahan chalega
    finally:
        elapsed = time.time() - start
        print(f"{label} khatam — {elapsed:.2f}s")

with timer_context("Database query"):
    results = db.query("SELECT ...")
# Database query shuru
# Database query khatam — 0.12s
```

Multiple Context Managers

```python
# Old way — nested
with open("input.txt") as infile:
    with open("output.txt", "w") as outfile:
        outfile.write(infile.read())

# Clean way — ek line!
with open("input.txt") as infile, open("output.txt","w") as outfile:
    outfile.write(infile.read())
```

❓ Interview Questions — Topic 7

Q: Context manager kya hota hai? Kyun use karte ho?
A: `__enter__` (setup) aur `__exit__` (cleanup) define karo. `with` statement ensure karta hai cleanup hamesha hoga — error aaye tab bhi. Resource leaks avoid karo. Java ka try-with-resources same concept.

Q: `__exit__` mein `return True` vs `False` kya karta hai?
A: `True` = exception suppress karo (caller ko nahi dikhegi). `False` = exception re-raise karo (default behavior). Mostly `False` return karo — exceptions hide karna dangerous hai.

Q: `@contextmanager` kab use karte ho class ke bajaye?
A: Simple resources ke liye — jahan ek `yield` se setup/teardown divide ho sake. Complex state management ho ya multiple methods chahiye toh class better hai.

📌 TOPIC 8: Exception Handling ⭐⭐⭐⭐

try / except / else / finally

```python
def safe_divide(a, b):
    try:
        result = a / b
    except ZeroDivisionError:
        print("Error: zero se divide nahi kar sakte")
        return None
    except TypeError as e:
        print(f"Wrong type: {e}")
        raise    # re-raise karo
    except (ValueError, OverflowError) as e:
        print(f"Value issue: {e}")   # multiple except ek saath
        return None
    except Exception as e:
        print(f"Unexpected: {type(e).__name__}: {e}")
        raise    # unknown errors always re-raise!
    else:
        print("Success!")   # koi exception nahi aaya toh
        return result
    finally:
        print("Always runs")   # cleanup code — hamesha
```

Custom Exceptions

```python
```

```python
# Base exception class
class AppError(Exception):
    pass

class ValidationError(AppError):
    def __init__(self, field, value, message):
        self.field   = field
        self.value   = value
        super().__init__(f"{field}={value}: {message}")

class NotFoundError(AppError):
    def __init__(self, resource, id_):
        super().__init__(f"{resource} with id {id_} nahi mila")

# Use karo
def get_user(user_id):
    if user_id <= 0:
        raise ValidationError("user_id", user_id, "positive hona chahiye")
    user = db.find(user_id)
    if not user:
        raise NotFoundError("User", user_id)
    return user

try:
    user = get_user(-1)
except ValidationError as e:
    print(f"Validation failed: {e.field}")   # custom attributes!
except NotFoundError as e:
    print(f"Not found: {e}")
except AppError as e:
    print(f"App error: {e}")   # catch all app errors
```

Exception Chaining

```python
try:
    data = read_from_db(user_id)
except DatabaseError as e:
    raise UserDataError(f"User {user_id} unavailable") from e
    # "from e" = original error preserve karo traceback mein
    # Debugging mein dono errors dikhenge
```

❓ Interview Questions — Topic 8

Q: `finally` block kab nahi chalta?

A: Agar process forcefully terminate ho jaaye (`os._exit()`), ya power off ho jaaye. Normal exceptions mein `finally` hamesha chalta hai — even `return` ke baad bhi.

Q: `raise` vs `raise e` kya difference hai?
A: `raise` — original exception re-raise karta hai (original traceback preserve). `raise e` — naya exception raise karta hai current location se (traceback change). Debugging ke liye `raise` prefer karo.

Q: Custom exception kyun banana chahiye?
A: Specific errors ko clearly identify karo. Caller different exceptions differently handle kar sake. Business logic errors ko system errors se alag karo. Custom attributes add kar sako (field name, value, etc).

📌 TOPIC 9: OOP — Classes aur Objects ⭐⭐⭐⭐

Class Basics

```python
```

```python
class BankAccount:
    # Class variable — sab instances share karte hain
    bank_name      = "National Bank"
    total_accounts = 0

    def __init__(self, owner, balance=0):
        # Instance variables — har object ka apna
        self.owner    = owner           # public
        self._balance = balance         # _ = private by convention
        self.__id     = id(self)        # __ = name mangling
        BankAccount.total_accounts += 1

    def deposit(self, amount):
        if amount > 0:
            self._balance += amount

    def withdraw(self, amount):
        if amount > self._balance:
            raise ValueError("Insufficient funds")
        self._balance -= amount

    @property
    def balance(self):          # getter
        return self._balance

    def __repr__(self):         # object ka string repr
        return f"BankAccount({self.owner}, {self._balance})"

    def __str__(self):          # print() ke liye
        return f"{self.owner} ka account: Rs. {self._balance}"

    def __eq__(self, other):    # == comparison
        return self.owner == other.owner

acc = BankAccount("Rahul", 5000)
acc.deposit(2000)
print(acc.balance)   # 7000
print(acc)           # Rahul ka account: Rs. 7000
```

Inheritance

```python
```

```python
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        raise NotImplementedError("Subclass must implement")   # abstract

    def info(self):
        return f"{self.name} says: {self.speak()}"

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)   # Java ka super() call
        self.breed = breed

    def speak(self):             # Override
        return "Woof!"

class Cat(Animal):
    def speak(self):             # Override
        return "Meow!"

animals = [Dog("Tommy", "Labrador"), Cat("Kitty")]
for animal in animals:
    print(animal.info())   # Polymorphism!
# Tommy says: Woof!
# Kitty says: Meow!

isinstance(Dog("x","y"), Animal)   # True
issubclass(Dog, Animal)            # True
```

Dunder / Magic Methods

```python
```

```python
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):    # v1 + v2
        return Vector(self.x+other.x, self.y+other.y)

    def __len__(self):           # len(v)
        import math
        return int(math.sqrt(self.x**2 + self.y**2))

    def __repr__(self):
        return f"Vector({self.x}, {self.y})"

v1 = Vector(1, 2)
v2 = Vector(3, 4)
print(v1 + v2)   # Vector(4, 6)
print(len(v2))   # 5
```

❓ Interview Questions — Topic 9

Q: `__init__` aur `__new__` mein kya fark hai?
A: `__new__` object create karta hai (memory allocate). `__init__` object ko initialize karta hai. Normally `__init__` hi use karo. `__new__` sirf tab override karo jab object creation khud control karni ho — singleton pattern, immutable types.

Q: `self` kya hota hai?
A: Method ka pehla argument jo current object refer karta hai. Java ka `this` jaisa. Python explicitly pass karna padta hai — har method mein. By convention `self` naam use karo.

Q: Multiple inheritance Python mein kaise kaam karta hai?
A: MRO (Method Resolution Order) — C3 linearization algorithm. `class C(A, B):` left to right, depth first. `.mro()` method se order dekho. Diamond problem handle hota hai. Python mein mixins ke liye common.

📌 TOPIC 10: Async / Await ⭐⭐⭐⭐⭐

Asynchronous programming — ek operation wait kare toh doosra kaam karo. I/O bound tasks ke liye — network calls, file I/O, DB queries. Thread se alag — single thread mein cooperative multitasking.

Sync vs Async — Time Difference

```python
```

```python
import time, asyncio

# SYNC — sequential, slow
def fetch_sync(url):
    time.sleep(2)   # network wait simulate
    return f"data from {url}"

# Total time: 3 * 2 = 6 seconds!
r1 = fetch_sync("api1")   # 2s wait
r2 = fetch_sync("api2")   # 2s wait
r3 = fetch_sync("api3")   # 2s wait

# ASYNC — concurrent, fast
async def fetch_async(url):
    await asyncio.sleep(2)   # yield control — don't block
    return f"data from {url}"

async def main():
    # Teeno simultaneously!
    r1, r2, r3 = await asyncio.gather(
        fetch_async("api1"),
        fetch_async("api2"),
        fetch_async("api3")
    )
    # Total time: ~2 seconds!

asyncio.run(main())
```

async def aur await Kya Karta Hai

```python
```

```python
# async def = coroutine function define karo
async def get_data(url):
    # await = "yahan pause karo, event loop ko control do"
    response = await http_client.get(url)   # async HTTP call
    data     = await response.json()        # async read
    return data

# Async function call karne ke tarike:

# 1. asyncio.run() — top level
result = asyncio.run(get_data("https://api.example.com"))

# 2. await — doosre async function ke andar
async def process():
    data = await get_data("https://api.example.com")
    return data

# NOTE: async function call karo toh coroutine milta hai
# Actual execute karne ke liye await ya asyncio.run() chahiye
coro   = get_data("url")           # sirf coroutine object — chala nahi!
result = asyncio.run(coro)         # ab chalega
```

asyncio.gather — Parallel Tasks

```python
```

```python
async def parallel_downloads():
    urls = ["url1", "url2", "url3"]

    # Sequential — slow
    results = []
    for url in urls:
        results.append(await fetch_async(url))   # 3 * 2s = 6s

    # Parallel — fast
    results = await asyncio.gather(
        *[fetch_async(url) for url in urls]   # all at once!
    )   # max(2s, 2s, 2s) = 2s

    # With error handling
    results = await asyncio.gather(
        fetch_async("url1"),
        fetch_async("url2"),
        return_exceptions=True   # crash nahi, error as value
    )
    for r in results:
        if isinstance(r, Exception):
            print(f"Failed: {r}")
```

Async vs Threading vs Multiprocessing

```python
```

```python
# ASYNC        — I/O bound, single thread, cooperative
# THREADING    — I/O bound, multiple threads, GIL limited
# MULTIPROCESSING — CPU bound, multiple processes, no GIL

# GIL kya hai?
# Global Interpreter Lock — ek waqt sirf ek Python thread execute.
# CPU bound parallel nahi. I/O bound mein GIL release — async works.

from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor

# Threading — I/O bound synchronous code ke liye
with ThreadPoolExecutor(max_workers=5) as executor:
    futures = [executor.submit(fetch_sync, url) for url in urls]
    results = [f.result() for f in futures]

# Multiprocessing — CPU bound
def cpu_heavy(n): return sum(i*i for i in range(n))

with ProcessPoolExecutor() as executor:
    results = list(executor.map(cpu_heavy, [10**6]*4))
```

❓ Interview Questions — Topic 10

Q: Async kyun use karte ho? Threading se better kyun?
A: I/O bound operations mein — network call, DB, file — thread block hota hai. Async mein `await` pe voluntarily yield karte hain, event loop doosra task chalata hai. Threads mein overhead aur context switching zyada. Async lightweight aur no race conditions (single thread).

Q: `await` kya karta hai exactly?
A: Coroutine ko pause karta hai current point pe. Event loop ko control deta hai doosra task chalane ke liye. Awaited operation complete hone pe same point se resume hota hai. CPU block nahi hota.

Q: GIL kya hai? Python mein parallel execution kaise karo?
A: Global Interpreter Lock — ek waqt ek Python thread execute. I/O bound = async (GIL release hota hai I/O pe). CPU bound = multiprocessing module (alag process, alag GIL).

Q: Event loop kya hota hai?
A: Async program ka traffic controller. Decide karta hai kaunsa coroutine kab chale. I/O wait pe dusra coroutine chalata hai. `asyncio.run()` event loop start karta hai.

📌 TOPIC 11: Type Hints aur TypedDict ⭐⭐⭐⭐

Basic Type Hints

```python
from typing import Optional, Union, List, Dict, Tuple, Callable, Literal

# Basic
name:   str   = "Rahul"
age:    int   = 25
score:  float = 0.85
active: bool  = True

# Collections
names:  List[str]           = ["Rahul", "Priya"]
config: Dict[str, int]      = {"timeout": 30}
coords: Tuple[float, float] = (28.6, 77.2)

# Optional — None ho sakta hai
email: Optional[str] = None      # str ya None

# Union — multiple types
id: Union[str, int] = "abc"      # string ya int

# Literal — specific values sirf
status: Literal["active","inactive","pending"] = "active"

# Callable — function type
handler: Callable[[str, int], bool]   # args=(str,int) return=bool

# Function annotations
def greet(name: str, age: int = 0) -> str:
    return f"{name}, age {age}"
```

> 💡 Type hints runtime mein enforce nahi hote — sirf hints hain. `mypy` tool static checking karta hai.

TypedDict — Structured Dictionary

```python
```

```python
from typing import TypedDict

# TypedDict = dict with defined structure
class UserProfile(TypedDict):
    name:  str
    age:   int
    email: str
    city:  str

# Optional fields ke liye total=False
class PartialUser(TypedDict, total=False):
    phone:   str    # optional
    address: str    # optional

# Use karo — dict ki tarah
user: UserProfile = {
    "name":  "Rahul",
    "age":   25,
    "email": "r@ex.com",
    "city":  "Delhi",
}

print(user["name"])   # "Rahul"
user["age"] = 26      # update

# TypedDict vs dict — kya fark?
# TypedDict = type checking support (mypy), documentation
# Runtime pe dict hi hai — koi overhead nahi
```

Protocols — Structural Subtyping

```python
```

```python
from typing import Protocol

class Drawable(Protocol):    # interface jaisa
    def draw(self) -> None: ...
    def area(self) -> float: ...

class Circle:
    def draw(self) -> None: print("Circle draw karo")
    def area(self) -> float: return 3.14 * self.r**2

# Circle explicitly Drawable inherit nahi karta
# But Drawable ka protocol satisfy karta hai — Duck typing + Types!
def render(shape: Drawable) -> None:
    shape.draw()
    print(f"Area: {shape.area()}")
```

❓ Interview Questions — Topic 11

Q: Type hints ka purpose kya hai? Runtime pe enforce hote hain?
A: Documentation + IDE support + static analysis tools (mypy). Runtime pe enforce nahi hote by default — sirf hints hain. Pydantic jaise libraries runtime validation ke liye type hints use karti hain.

Q: TypedDict vs dataclass vs Pydantic — kab kaunsa?
A: TypedDict = existing dict-based code mein type safety add karo, no overhead. Dataclass = simple data container, auto `__init__`, `__repr__`. Pydantic = validation + serialization + FastAPI integration.

📌 TOPIC 12: Pydantic v2 ⭐⭐⭐⭐⭐

Data validation library — type hints se automatically validate, parse, aur serialize karta hai. FastAPI + modern Python mein everywhere use hota hai.

Basic Model

```python
```

```python
from pydantic import BaseModel, Field, field_validator
from typing import Optional, List

class UserProfile(BaseModel):
    name:   str
    age:    int
    email:  str
    city:   str           = "Unknown"   # default
    skills: List[str]     = []          # default
    score:  Optional[float] = None

# Valid data
user = UserProfile(name="Rahul", age=25, email="r@ex.com")
print(user.name)          # "Rahul"
print(user.city)          # "Unknown" — default
print(user.model_dump())  # dict mein convert

# Type coercion — Pydantic smart hai
user2 = UserProfile(name="Priya", age="25", email="p@ex.com")
print(type(user2.age))    # <class 'int'> — string se convert!

# Invalid data — ValidationError
# UserProfile(name="X", age="not_a_number", email="x@x.com")
# pydantic.ValidationError: 1 validation error
```

Field + Validators

```python
```

```python
from pydantic import BaseModel, Field, field_validator, model_validator

class Product(BaseModel):
    name:     str   = Field(min_length=2, max_length=100)
    price:    float = Field(gt=0, lt=1_000_000)    # > 0, < 1M
    quantity: int   = Field(ge=0, le=9999)          # >= 0
    category: str   = Field(description="Product category")
    discount: float = Field(default=0.0, ge=0.0, le=1.0)

    @field_validator("name")
    @classmethod
    def name_must_not_be_empty(cls, v):
        if v.strip() == "":
            raise ValueError("Name empty nahi ho sakta")
        return v.strip()   # cleaned value return karo

    @field_validator("category")
    @classmethod
    def valid_category(cls, v):
        allowed = ["Electronics", "Clothing", "Food", "Books"]
        if v not in allowed:
            raise ValueError(f"Category must be one of {allowed}")
        return v

    @model_validator(mode="after")   # sab fields validate hone ke baad
    def check_discount_logic(self):
        if self.discount > 0.5 and self.price < 100:
            raise ValueError("Cheap items pe 50% se zyada discount nahi")
        return self
```

Serialization

```python
```

```python
user = UserProfile(name="Rahul", age=25, email="r@ex.com")

# Pydantic → dict
d = user.model_dump()
# {"name": "Rahul", "age": 25, "email": "r@ex.com", ...}

# Pydantic → JSON string
j = user.model_dump_json()
# '{"name":"Rahul","age":25,...}'

# dict → Pydantic
data  = {"name": "Priya", "age": 30, "email": "p@ex.com"}
user2 = UserProfile(**data)
user3 = UserProfile.model_validate(data)   # same

# JSON → Pydantic
user4 = UserProfile.model_validate_json('{"name":"Amit",...}')
```

❓ Interview Questions — Topic 12

Q: Pydantic TypedDict se better kyun hai?
A: TypedDict = sirf type hints, runtime validation nahi. Pydantic = type hints + runtime validation + type coercion + Field constraints + custom validators + serialization built-in.

Q: `@field_validator` aur `@model_validator` mein kya fark?
A: `@field_validator` = specific field pe validation. `@model_validator` = multiple fields ek saath validate karo (cross-field logic). `model_validator mode="after"` = sab fields validate hone ke baad.

Q: `model_dump()` kab use karte ho?
A: Pydantic object ko dict mein convert karna ho — API response, logging, storage, JSON serialization ke liye.

📌 TOPIC 13: map(), filter(), zip(), enumerate() ⭐⭐⭐

map() — Har Item Pe Function Apply Karo

```python
```

```python
nums = [1, 2, 3, 4, 5]

doubled = list(map(lambda x: x*2, nums))   # [2,4,6,8,10]

# List comprehension se same — preferred!
doubled = [x*2 for x in nums]   # ← zyada readable

# Named function ke saath
def celsius_to_fahrenheit(c): return c*9/5 + 32
temps_f = list(map(celsius_to_fahrenheit, [0, 25, 37, 100]))
```

filter() — Condition Pe Filter Karo

```python
scores = [0.9, 0.4, 0.7, 0.3, 0.8]

high = list(filter(lambda s: s > 0.7, scores))   # [0.9, 0.8]

# Comprehension preferred:
high = [s for s in scores if s > 0.7]   # cleaner!

# None se filter — falsy values hatao
mixed = [1, None, 2, "", 3, False, 4]
clean = list(filter(None, mixed))        # [1, 2, 3, 4]
```

zip() — Do Lists Parallel Iterate

```python
```

```python
names  = ["Rahul", "Priya", "Amit"]
scores = [90, 85, 78]
cities = ["Delhi", "Mumbai", "Pune"]

for name, score in zip(names, scores):
    print(f"{name}: {score}")

# Dict banana
result = dict(zip(names, scores))
# {"Rahul":90, "Priya":85, "Amit":78}

# Unzip — zip(*) se
pairs = [(1,"a"), (2,"b"), (3,"c")]
nums, letters = zip(*pairs)
# nums=(1,2,3)  letters=("a","b","c")

# zip_longest — alag lengths ke liye
from itertools import zip_longest
list(zip_longest([1,2,3],[4,5],fillvalue=0))   # [(1,4),(2,5),(3,0)]
```

enumerate() — Index + Value

```python
fruits = ["apple", "banana", "cherry"]

for i, fruit in enumerate(fruits):
    print(f"{i}: {fruit}")
# 0: apple
# 1: banana

# Start index
for i, fruit in enumerate(fruits, start=1):
    print(f"{i}. {fruit}")   # 1. apple, 2. banana ...

# dict mein convert
indexed = {i: fruit for i, fruit in enumerate(fruits)}
```

❓ Interview Questions — Topic 13

Q: `map` vs list comprehension — kaunsa prefer karo?
A: List comprehension zyada Pythonic aur readable hai. `map()` legacy code mein milta hai. Performance similar. Comprehension prefer karo — zyada clear intent.

Q: `zip()` kya karta hai jab lists alag length ki hon?
A: Shortest list tak iterate karta hai — extra elements ignore. `zip_longest()` use karo alag

lengths ke liye `fillvalue` ke saath.

📌 TOPIC 14: String Formatting aur Methods ⭐⭐⭐

f-String — Sabse Important

```python
name  = "Rahul"
score = 0.856789

# Basic
f"Hello {name}!"             # "Hello Rahul!"
f"2 + 2 = {2 + 2}"           # "2 + 2 = 4"
f"{name.upper()} rocks!"     # "RAHUL rocks!"

# Format specifiers
f"{score:.2f}"               # "0.86"       — 2 decimal places
f"{score:.1%}"               # "85.7%"      — percentage
f"{1000000:,}"               # "1,000,000"  — comma separator
f"{name:>10}"                # "     Rahul" — right align
f"{name:<10}"                # "Rahul     " — left align
f"{name:^10}"                # "  Rahul   " — center align

# Multi-line
report = f"""
Name:  {name}
Score: {score:.1%}
Grade: {"Pass" if score >= 0.5 else "Fail"}
"""
```

Important String Methods

```python
```

```python
text = "  Hello, World!  "

# Cleaning
text.strip()            # "Hello, World!"   — both ends
text.lstrip()           # "Hello, World!  " — left only
text.rstrip()           # "  Hello, World!" — right only

# Case
text.upper()            # "HELLO, WORLD!"
text.lower()            # "hello, world!"
text.title()            # "Hello, World!"

# Search
text.find("World")      # 8 — index, -1 agar nahi mila
text.count("l")         # 3
"World" in text         # True

# Modify
text.replace("World","Python")  # "  Hello, Python!  "
",".join(["a","b","c"])         # "a,b,c"
"a,b,c".split(",")              # ["a", "b", "c"]

# Check
"123".isdigit()         # True
"abc".isalpha()         # True
"abc123".isalnum()      # True
```

❓ Interview Questions — Topic 14

Q: f-string Python 2 ke `.format()` se better kyun?
A: f-string zyada readable — variable directly andar. Faster bhi hai. `.format()` mein positional `{0}` ya named `{name}` use karna padta. f-string Python 3.6+ mein available.

Q: String immutable hai — performance pe kya effect?
A: Har concatenation naya string banata hai. Large loops mein `+` se join karna $O(n^2)$ ho sakta hai. `"".join(parts)` prefer karo — sab pieces ek baar mein join. Zyada efficient.

📌 TOPIC 15: itertools aur functools ⭐⭐⭐

itertools — Powerful Iterators

```python
```

```python
import itertools

# chain — multiple iterables ko ek mein
list(itertools.chain([1,2],[3,4],[5,6]))   # [1,2,3,4,5,6]

# product — Cartesian product (nested loops replace)
list(itertools.product([1,2],[3,4]))
# [(1,3),(1,4),(2,3),(2,4)]

# combinations — unique groups
list(itertools.combinations("ABCD", 2))
# [("A","B"),("A","C"),("A","D"),("B","C"),("B","D"),("C","D")]

# permutations — ordered arrangements
list(itertools.permutations("ABC", 2))
# [("A","B"),("A","C"),("B","A"),("B","C"),("C","A"),("C","B")]

# groupby — group by key (sorted hona chahiye pehle!)
data = [("A",1),("A",2),("B",3),("B",4)]
for key, group in itertools.groupby(data, key=lambda x: x[0]):
    print(key, list(group))
# A [("A",1),("A",2)]
# B [("B",3),("B",4)]

# islice — lazy slicing
gen     = itertools.count(0)   # infinite counter
first10 = list(itertools.islice(gen, 10))   # [0..9]
```

functools — Higher-Order Functions

```python
```

```python
from functools import reduce, partial, lru_cache, wraps

# reduce — list ko ek value mein
product = reduce(lambda a, b: a*b, [1,2,3,4,5])   # 120
total   = reduce(lambda a, b: a+b, [1,2,3,4,5])   # 15

# partial — function arguments pre-fill karo
def power(base, exp): return base ** exp
square = partial(power, exp=2)   # exp=2 fix karo
cube   = partial(power, exp=3)   # exp=3 fix karo
square(5)   # 25
cube(3)     # 27

# lru_cache — memoization (expensive computations cache karo)
@lru_cache(maxsize=128)
def fibonacci(n):
    if n < 2: return n
    return fibonacci(n-1) + fibonacci(n-2)

fibonacci(50)   # fast — cached!
fibonacci.cache_info()   # CacheInfo(hits=48, misses=51, ...)

# Unlimited cache
@lru_cache(maxsize=None)
def expensive_calc(n): ...
```

❓ Interview Questions — Topic 15

Q: `lru_cache` kya hai aur kab use karte ho?
A: Memoization decorator — function results cache karta hai. Baar baar same arguments se call hone wale expensive functions ke liye. Recursive functions (fibonacci, dp) pe dramatic speedup. `maxsize=None` se unlimited cache.

Q: `reduce()` ka example do.
A: `reduce(lambda a,b: a+b, [1,2,3,4,5])` = 15. List ko ek value mein collapse karta hai. Java ka `stream().reduce()` jaisa.

📌 TOPIC 16: File I/O, Modules, Packages ⭐⭐⭐

File Reading/Writing

```python
```

```python
# Read
with open("data.txt", "r", encoding="utf-8") as f:
    content = f.read()              # poora file
    # ya
    lines   = f.readlines()         # list of lines
    # ya (memory efficient)
    for line in f:                  # ek ek line
        process(line.strip())

# Write
with open("output.txt", "w", encoding="utf-8") as f:
    f.write("Hello World\n")
    f.writelines(["line1\n", "line2\n"])

# Append
with open("log.txt", "a") as f:
    f.write("New log entry\n")

# JSON
import json

data = {"name": "Rahul", "scores": [90, 85, 78]}
with open("data.json", "w") as f:
    json.dump(data, f, indent=2)

with open("data.json") as f:
    loaded = json.load(f)

# JSON string ↔ dict
json_str = json.dumps(data)        # dict → JSON string
obj      = json.loads(json_str)    # JSON string → dict
```

Modules aur Imports

python

```python
# Standard imports
import os, sys, json, math

# Specific import
from os import path, getcwd
from datetime import datetime, timedelta

# Alias
import numpy as np
import pandas as pd

# Relative import (package ke andar)
from . import utils            # same package
from ..models import User      # parent package

# __name__ check — script directly chalaya ya import kiya?
if __name__ == "__main__":     # directly chalaya
    main()

# Package structure
# myapp/
#   __init__.py    ← package marker
#   models.py
#   utils.py
#   tests/
#     __init__.py
#     test_models.py
```

os aur pathlib

```python
```

```python
import os
from pathlib import Path

# os module
os.getcwd()                          # current directory
os.listdir(".")                      # files list
os.path.exists("file.txt")           # file exist karta hai?
os.path.join("dir", "file.txt")      # path join
os.makedirs("new/dir", exist_ok=True)   # directory banana
os.environ.get("HOME", "/home")      # environment variable

# pathlib — modern way (Python 3.4+)
p = Path("data/file.txt")
p.exists()           # True/False
p.is_file()          # True if file
p.suffix             # ".txt"
p.stem               # "file"
p.parent             # Path("data")
p.read_text()        # file read karo
p.write_text("hi")   # file write karo
list(Path(".").glob("*.py"))   # pattern match
```

❓ Interview Questions — Topic 16

Q: `__name__ == "__main__"` kyun use karte ho?
A: File directly run karo toh `__name__ == "__main__"`. Import karo toh module name hota hai. Main logic guard karo — agar import kiya toh execute mat karo. Test aur reusability ke liye.

Q: `os.path` vs `pathlib` — kaunsa prefer karo?
A: `pathlib` modern aur object-oriented hai — Python 3.6+. `p / "subdir" / "file.txt"` se path join kar sako. `os.path` string based, older code mein milega. New code mein `pathlib` prefer karo.

📌 TOPIC 17: Python Internals (Senior Level) ⭐⭐⭐⭐

Memory Management

```python
```

```python
# Reference Counting
import sys
a = [1, 2, 3]
sys.getrefcount(a)   # 2 (a + getrefcount argument)

b = a           # reference count +1
del b           # reference count -1
# count = 0 hone pe garbage collected

# id() — memory address
x = 42
y = 42
id(x) == id(y)   # True — Python caches small ints!

# Integer caching — -5 to 256 same object
a = 100; b = 100; a is b   # True
a = 300; b = 300; a is b   # False — new objects

# String interning
s1 = "hello"; s2 = "hello"
s1 is s2   # True — Python interns string literals

# Garbage collection — circular references ke liye
import gc
gc.collect()   # manually collect
```

GIL — Global Interpreter Lock

```python
```

```python
# GIL = ek waqt sirf ek Python thread CPython mein

# I/O bound — GIL release hota hai wait mein
import threading
def download(url): ...   # I/O bound
threads = [threading.Thread(target=download, args=(u,)) for u in urls]
for t in threads: t.start()   # effective for I/O

# CPU bound — GIL problem → multiprocessing use karo
from multiprocessing import Pool
def heavy_calc(n): return sum(i*i for i in range(n))

with Pool() as pool:            # alag processes — no GIL
    results = pool.map(heavy_calc, [10**6]*4)

# NumPy — C level, GIL release karta hai
import numpy as np
arr = np.array([1,2,3,4,5])
np.sum(arr**2)   # C mein run — fast, GIL nahi rokata
```

slots — Memory Optimization

```python
# Normal class — har instance ek __dict__ rakhta hai (overhead)
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

# __slots__ — fixed attributes, no __dict__
class PointFast:
    __slots__ = ["x", "y"]   # allowed attributes fix karo
    def __init__(self, x, y):
        self.x = x
        self.y = y

import sys
sys.getsizeof(Point(1,2))     # ~56 bytes + dict overhead
sys.getsizeof(PointFast(1,2)) # ~48 bytes — smaller!

# Lakhs of objects banane ho toh memory matter karta hai
# __slots__ se dynamic attribute add nahi kar sakte
```

copy vs deepcopy

```python
import copy

original = {"name": "Rahul", "scores": [90, 85, 78]}

# Assignment — same object
alias = original
alias["name"] = "Priya"         # original bhi change!

# Shallow copy — outer copy, inner shared
shallow = copy.copy(original)
shallow["name"] = "Priya"       # original change nahi
shallow["scores"].append(95)    # original["scores"] bhi change!

# Deep copy — completely independent
deep = copy.deepcopy(original)
deep["scores"].append(95)       # original bilkul safe
```

❓ Interview Questions — Topic 17

Q: Python memory management kaise kaam karta hai?
A: Reference counting — har object ka count. Zero hone pe freed. Circular references ke liye cyclic garbage collector. CPython mein small integers (-5 to 256) aur string literals cached hote hain.

Q: Shallow copy vs deep copy kab use karo?
A: Shallow = outer level copy — andar wale objects shared. Fast aur mostly sufficient. Deep = completely independent copy — nested structures ke liye. `copy.deepcopy()` use karo jab modifications original ko affect na kare.

Q: `__slots__` kab use karte ho?
A: Bahut saare instances banane ho — millions of objects. Memory optimization ke liye fixed attributes define karo. Trade-off: dynamic attributes add nahi kar sakte, inheritance tricky.

📋 QUICK REVISION CHEATSHEET

Topic	Key Points
Data Types	str=immutable, list=mutable ordered, dict=key-value ordered(3.7+), tuple=immutable list, set=unique unordered
Dynamic Typing	Type runtime pe decide. `type()` se check. `isinstance()` preferred. Java se alag — koi declare nahi.

Topic	Key Points
*args /**kwargs	`*args` → tuple. `**kwargs` → dict. `*list` unpack. `**dict` unpack when calling.
Mutable Default Bug	`def f(lst=[])` ← WRONG! `def f(lst=None)` then `lst=[]` inside ← CORRECT
Comprehension	`[expr for x in iter if cond]` | `{k:v for ...}` | `{expr}` set | `(expr)` generator
Decorator	`@dec = func=dec(func)`. `@wraps` preserve naam. `@property` getter/setter. `@staticmethod` no self. `@classmethod` cls milta.
Generator	`yield` = pause karo. Memory efficient. StopIteration on exhaust. `(x for x)` = lazy.
Context Manager	`__enter__` setup, `__exit__` cleanup. Always cleanup. `@contextmanager` decorator.
Exceptions	Specific first, broad last. `raise` re-raise. `raise e from orig` chain. Custom exception: inherit Exception.
OOP	`__init__` constructor, `self = this`. `_` private convention, `__` name mangling. `super()` parent call.
Async	`async def` coroutine. `await` pause. `asyncio.gather()` parallel. GIL: I/O=async, CPU=multiprocessing.
copy	`=` same object. `copy.copy()` shallow. `copy.deepcopy()` independent.
itertools	chain, product, combinations, permutations, groupby, islice
functools	`@lru_cache` memoize. `partial` pre-fill args. `reduce` collapse to one. `@wraps` preserve metadata.

Java vs Python — Quick Reference

Java	Python
`null`	`None`
`true` / `false`	`True` / `False`
`int x = 5`	`x = 5`

Java	Python
`String s = "hi"`	`s = "hi"`
`ArrayList<T>`	`list`
`HashMap<K,V>`	`dict`
`for(T x: list)`	`for x in list:`
try-with-resources	`with` statement
`@Override`	Decorator / override method
`List<String>`	`List[str]` (type hint)
`Optional<T>`	`Optional[str]`
`x -> x*2`	`lambda x: x*2`
`stream().map().filter()`	`[x*2 for x in lst if cond]`
`static void method()`	`@staticmethod`
`CompletableFuture`	`async/await`
`Integer.MAX_VALUE`	No overflow — unlimited int!
`String.format()`	f-string

Python Complete Interview Prep Guide | 5.5 YOE Java Dev → Python Expert | Topics 1–17 Covered